DESIGN AND DEVELOP A REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

SOMA PRATHIBHA (VTU24779)

10211CS224
Full Stack Application Development
Slot : E3-B19

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”Design and Develop a React JS Based Web Application For Ticket Booking of an Internal Department Event” by Soma Prathibha (VTU24779) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.X.Arogya Presskila

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. M. Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Soma Prathibha)

Date:06/05/2026

ABSTRACT

The project titled ”Design and Development of a React JS Web Application for Ticket Booking of an Internal Department Event” aims to develop an efficient and user-friendly web platform for managing ticket bookings for departmental events such as technical fests, seminars, workshops, symposiums, and conferences [1]. This application is developed using React JS for the frontend to create an interactive and responsive user interface featuring dedicated user and admin dashboards. The backend is implemented using Node.js and Express.js to handle server-side logic and RESTful API routing, while MySQL is utilized as the database for securely storing structured data such as user credentials, event details, and real-time booking records [5, 3].

Keywords: React JS, Event Management, Ticket Booking, Full Stack Development, Node.js, MySQL.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JS	JavaScript
Full Stack	React.js, Express.js, Node.js, MySQL
UI	User Interface

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

Manual processes for managing internal department events like fests, seminars, and workshops are time-consuming, hard to track, and prone to errors. To solve this, a web-based ticket booking and management system has been developed. The application provides a user-friendly interface where students can register, log in, view events, and book tickets easily using a responsive React.js frontend [7]. Event administrators have access to a dashboard to create events, track registrations, and manage bookings efficiently. The backend is built with Node.js and Express.js, while MySQL is used for secure data storage [4]. This full-stack architecture ensures reliability and performance even with high user activity. Overall, the platform streamlines event management, reduces manual effort, prevents overbooking, and improves accessibility for the academic community.

1.2 Aim of the Project

The primary objective of this project is to build a web application using React JS for booking tickets for internal department events [1]. The system is intended to offer a simple and efficient platform for students to explore event information and reserve tickets online. It also seeks to minimize manual registration efforts, enhance event coordination, and deliver a user-friendly experience for both participants and administrators.

1.3 Scope of the Project

The scope of this project is to design and implement a React JS web application that enables online ticket booking for internal department events [6]. The system aims to streamline the event registration process and enhance the management of participant data within an academic environment. The application includes functionalities such as user sign-up, secure login, event exploration, and ticket reservation. Users can browse upcoming events, access detailed information, and book tickets through an intuitive interface. In addition, administrators are provided with tools to create and manage events, as well as track booking activities.

1.4 Framework

The framework of this project is structured to handle ticket booking for internal department events in a clear and systematic manner. The application is built using React JS, ensuring a dynamic and responsive user interface. It consists of various modules, including user registration, authentication, event listing, ticket reservation, and administrative control. These components work together to deliver a seamless booking experience. The system follows a component-based approach, where different sections of the application are interconnected to efficiently manage user interactions and event-related operations [7].

Figure 1.1: Framework architecture [1]

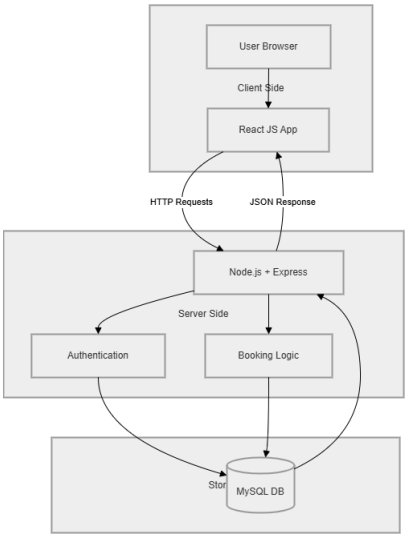

Figure 1.1: Shows the system architecture where React frontend communicates with Node.js backend and MySQL database via REST APIs.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

2.1 Existing Event Booking Platforms

1. Cvent: A powerful event management tool for large corporate events and conferences, offering features like registration and attendee tracking [8]. However, it is costly and overly complex for small departmental use.

2. Meetup: A platform for organizing community-based events and group activities [9]. It is simple to use but lacks advanced ticketing and administrative features needed for academic events.

3. CampusGroups: A university-level platform for managing student organizations and events, with features like registration and analytics [10]. It requires institutional access and is not suited for individual departments.

4. Whova: An event platform focused on networking and engage-

ment, commonly used for conferences [11]. Its advanced features and pricing make it less suitable for simple college events.

2.2 Comparative Analysis of Booking Frameworks

The table 2.1 highlights the differences between conventional event management methods and the proposed application based on operational capabilities.

Table 2.1: Evaluation of Conventional Methods versus Proposed Application [6]

Capabilities	**Conventional Methods**	**Proposed Application**
System Accessibility	Restricted Access	Web Based Platform
Data Processing	Batch Processing	Instant Updates
Administrative Tools	Manual Tracking	Centralized Dashboard
Architecture Style	Monolithic Structure	Microservices Approach
Booking Tracking	Paper Based Records	Digital History Logs
Security Measures	Standard Access	JWT Authentication

Table 2.1: Compares traditional manual methods with the proposed web-based system across accessibility, processing, administration, architecture, tracking, and security parameters.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The environment setup is the initial step required to develop the frontend of the application. The project is created using React JS. Node.js and npm are required to install packages and run the application environment [1].

Requirements: Node.js, npm, Visual Studio Code, React JS

Commands Used:

```
npm create vite@latest frontend
cd frontend
npm install
npm run dev
```

3.1.2 Structure of the Project

The frontend project follows a component based folder structure to maintain the application efficiently. The React JS application is di-

vided into specific files based on functionality, including dedicated pages for user and admin workflows, along with service integration files. This structure helps in easy navigation, better code management, and improved maintainability [1].

The main project structure includes the following key components and utility files:

- App.js – Main component that manages routing and application state
- api.js & authService.js – Utility files handling REST API calls and token management
- Home.jsx & EventsPage.jsx – Components for displaying the main interface and event catalogs
- AdminDashboard.jsx & UserDashboard.jsx – Dedicated role based control panels
- BookingPage.jsx & BookingHistory.jsx – Components managing the ticket reservation flow
- styles.css – Contains global and component specific styling for the project

3.1.3 Execution Procedure

The execution procedure explains the steps required to run the React JS frontend application successfully.

1. Open the frontend project folder in Visual Studio Code.
2. Open the terminal in the project directory.
3. Install the required dependencies using npm.
4. Execute the development command and open the generated localhost link in the browser to view the application.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment setup is required to manage data handling, security, and application logic. Unlike a traditional monolithic backend, this application utilizes a highly scalable microservices architecture built with Java and Spring Boot [3].

To set up the backend environment, multiple independent service folders are maintained. Java Development Kit (JDK) and Maven are required to build and run these services.

Requirements: Java Development Kit (JDK 17+), Apache Maven, Spring Boot, Visual Studio Code or IntelliJ IDEA

Commands Used (for each service):

```
mvn clean install
mvn spring-boot:run
```

3.2.2 Structure of the Project

The backend project follows a distributed microservices architecture to manage server side functionality efficiently. The structure separates different domains into standalone applications. This improves scalability, fault tolerance, and independent deployment capabilities [4].

The backend infrastructure consists of the following modular services:

- eureka-server/ – Acts as the service registry for dynamic discovery of all microservices
- api-gateway/ – Routes incoming frontend requests to the appropriate backend service
- identity-service/ – Handles user authentication, registration, and JWT token generation
- user-service/ – Manages user profile data and role based access control
- order-service/ – Manages event ticket bookings, reservation logic, and order history

3.2.3 Execution Procedure

Due to the microservices architecture, the services must be started in a specific sequence to ensure proper registration and routing.

1. Open the backend project workspace in your IDE.
2. Start the Eureka Server first to initialize the service registry.
3. Start the API Gateway to enable request routing.
4. Start the remaining microservices (Identity Service, User Service, Order Service).
5. Verify that all services are successfully registered on the Eureka Server dashboard.

Figure 3.1: Backend Microservices Architecture [3]

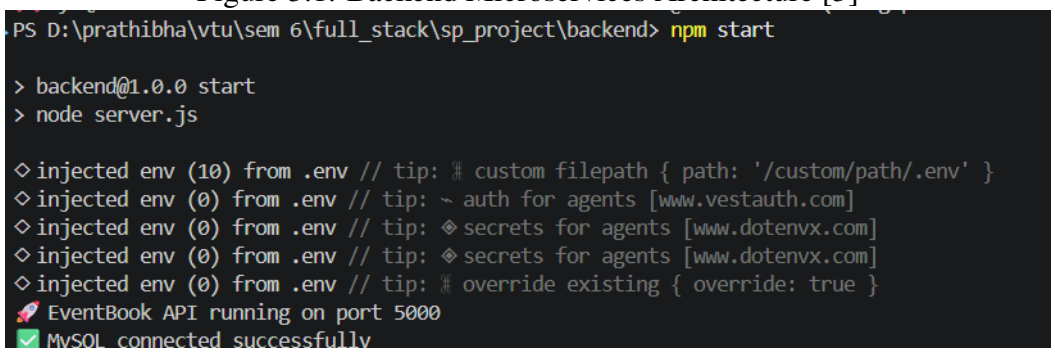

Figure 3.1: Illustrates the microservices setup including Eureka Server, API Gateway, Identity, User, and Order services for scalable backend operations.

3.3 Database Implementation

3.3.1 Database Configuration

The database is configured to store all required information related to the ticket booking system. Given the microservices architecture, MySQL is utilized to maintain records of users, events, and bookings securely [5]. Proper configuration ensures efficient data access and data consistency across services.

The database stores:

1. User registration and authentication details
2. Event catalogs and capacity information
3. Booking transactions and order histories
4. Admin related configuration records

3.3.2 Schema Structure

The schema structure defines how data is organized inside the relational database. Different tables are created to store specific information related to users, events, and ticket orders [5].

The main schema includes:

1. Users Schema (Managed by Identity and User Services)
2. Events Schema (Managed by Admin controls)

3. Orders Schema (Managed by Order Service)

3.3.3 Tables Created

User Table

- User ID (Primary Key)
- Name
- Email
- Password (Encrypted)
- Role (User/Admin)

Event Table

- Event ID (Primary Key)
- Event Name
- Date and Time
- Venue
- Description and Available Capacity

Booking Table

- Booking ID (Primary Key)
- User ID (Foreign Key)

- Event ID (Foreign Key)
- Booking Date
- Status (Confirmed/Pending)

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project architecture outlines the structural design of the ticket booking system. The application follows a client-server model where users interact through a React-based frontend, while the backend handles request processing and communicates with the MySQL database [1, 2].

Architecture Flow:

Architecture Flow:

Figure 4.1: Project Architecture [6]

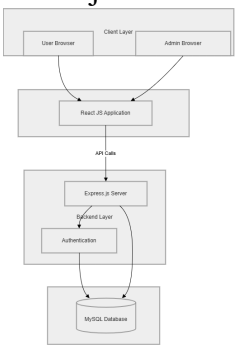

Figure 4.1: This figure illustrates the client-server architecture, where the React frontend communicates with the Node.js/Express backend, which interacts with the MySQL database for data processing and storage.

4.2 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how data moves across different components of the system. It highlights the interaction between users, application modules, and the database. Within this system, users can register, log in, explore events, and book tickets. All user inputs are processed and stored efficiently for future use [7].

Figure 4.2: Data Flow Diagram [6]

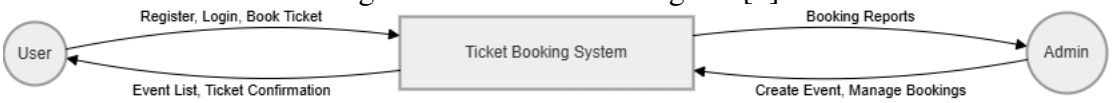

Figure 4.2: Maps data flow between users, admins, system modules, and database showing registration, authentication, and booking processes.

4.3 Module 1: User Management Module

This module is responsible for handling all user-related operations. It enables users to register, authenticate, and access the application features. It also ensures secure storage of user data and maintains active sessions for a seamless user experience.

Functions included:

- User Registration
- Login Authentication

- Profile Management
- Session Handling

This module ensures safe and controlled access to the system.

4.4 Module 2: Event Booking Module

This module manages the process of viewing and reserving event tickets. Users can browse through available events, check detailed information, and proceed with booking.

Functions included:

- Event Listing
- Ticket Reservation
- Booking Confirmation
- Viewing Event Details

It ensures a smooth and efficient booking workflow for users.

4.5 Module 3: Ticket Booking

This module represents the core functionality of the application. It allows users to select events, choose ticket types or seats, and finalize their bookings. The system updates ticket availability in real time

by interacting with the MySQL database, thereby preventing duplicate bookings and ensuring data consistency [5].

4.5.1 Advantages of this Project

a) Easy to Use Interface:

The application offers a clean and user-friendly interface, making it simple for users to navigate, explore events, and complete bookings.

b) Real-Time Updates:

Ticket availability is updated instantly, reducing the chances of over-booking and ensuring accurate information.

4.5.2 Disadvantages

- Requires a stable internet connection for proper functioning.
- Performance may degrade if the system handles a large number of users simultaneously.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Home Page

The Home Page provides an interface for new users to register by entering details such as name, email, department, and password. Once registration is completed, users can log in and access the event booking platform.

Figure 5.1: Home Page [1]

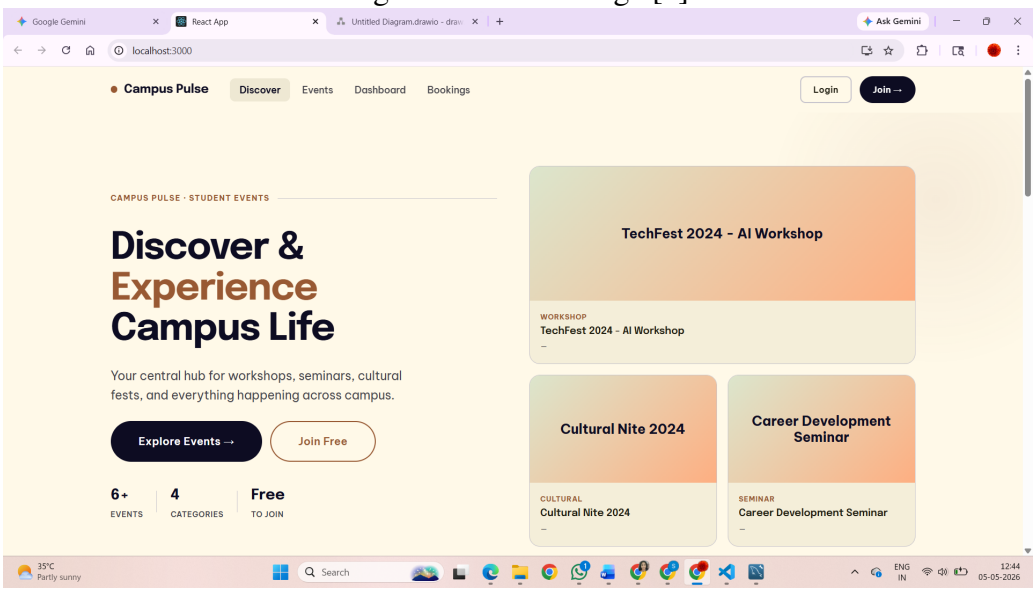

Figure5.1: Displays the registration interface where new users enter

personal details to create an account.

5.2 Login Page

The Login Page enables registered users to securely enter the system. Users must provide valid email and password credentials. Upon successful verification, they are redirected to the event listing page.

This page ensures controlled access by preventing unauthorized users from entering the system.

Figure 5.2: User Login Page [1]

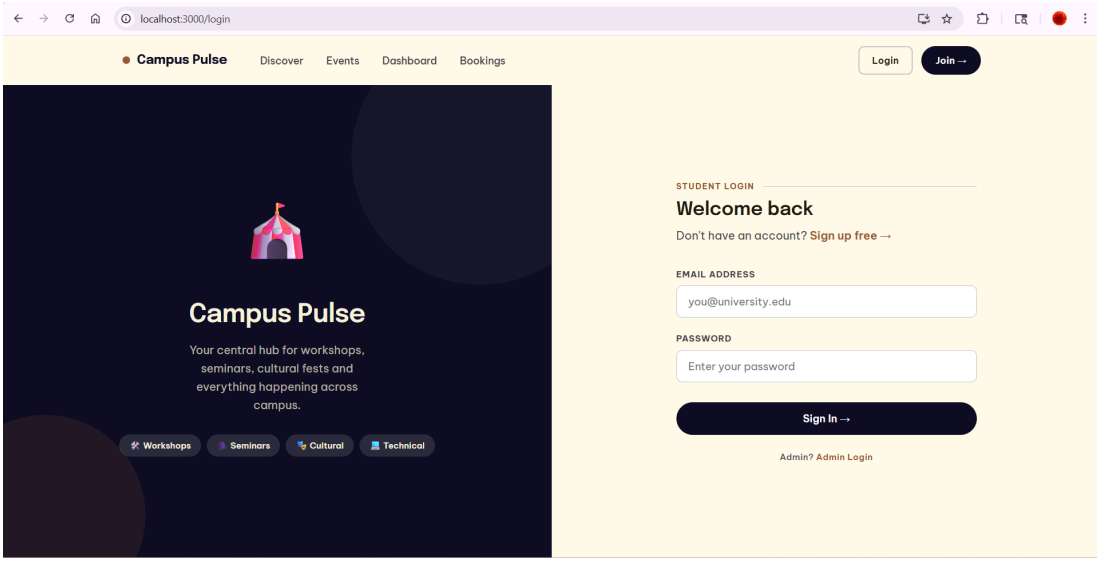

Figure 5.2: Shows the secure authentication page where registered users enter credentials to access the system.

Working

- Users input their registered email and password.

- The system verifies the entered credentials.
- If valid, the user is redirected to the event listing page.
- If invalid, an appropriate error message is shown.

5.3 Event Listing Page

The Event Listing Page presents all available events retrieved from the database. After logging in, users are directed to this page, where they can view details such as event name, date, venue, and description.

Figure 5.3: Event Listing Page [6]

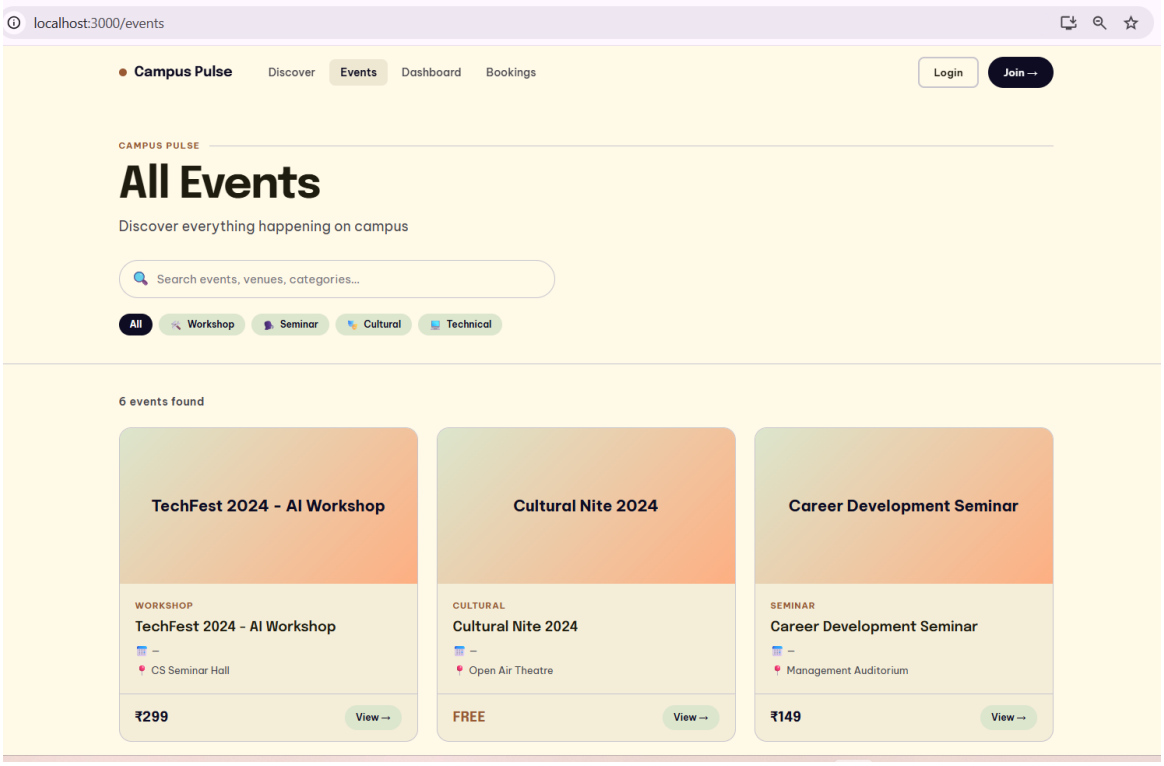

Figure 5.3: Displays all available events with details like name, date, venue, and a ”Book Ticket” option for each.

Working

- Events are displayed in a structured format.
- Each event includes a ”Book Ticket” option.
- Users can proceed directly to ticket booking.

5.4 Ticket Reservation and Confirmation Module

This module allows users to reserve tickets for selected events. When a user chooses an event, the system processes the booking request and saves the details in the MySQL database. After successful booking, a confirmation message along with a unique ticket ID is generated.

Figure 5.4: Ticket Reservation and Confirmation Module [5]

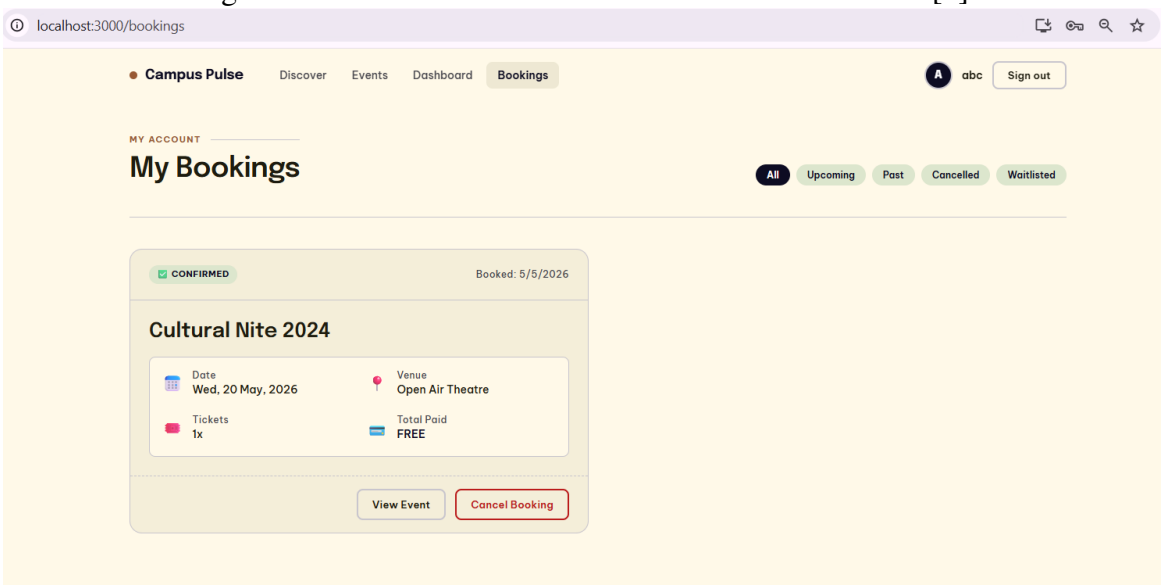

Figure 5.4: Shows booking summary and generates unique ticket ID upon successful reservation confirmation.

Working

- User selects an event from the listing page.
- Booking request is sent to the backend server.
- The server validates and stores the data in the database.
- A confirmation message with a unique ticket ID is generated.
- The user can view or download the ticket.

5.5 Admin Module

The Admin Module allows authorized personnel to manage the system efficiently. It includes secure login functionality and tools to create, update, and monitor events. Administrators can add event details such as name, date, venue, and description, as well as track user bookings.

5.5.1 Admin Login Page

The Admin Login Page provides restricted access to the administrative dashboard. Only valid credentials allow entry, ensuring system security.

Figure 5.5: Admin Login Page [1]

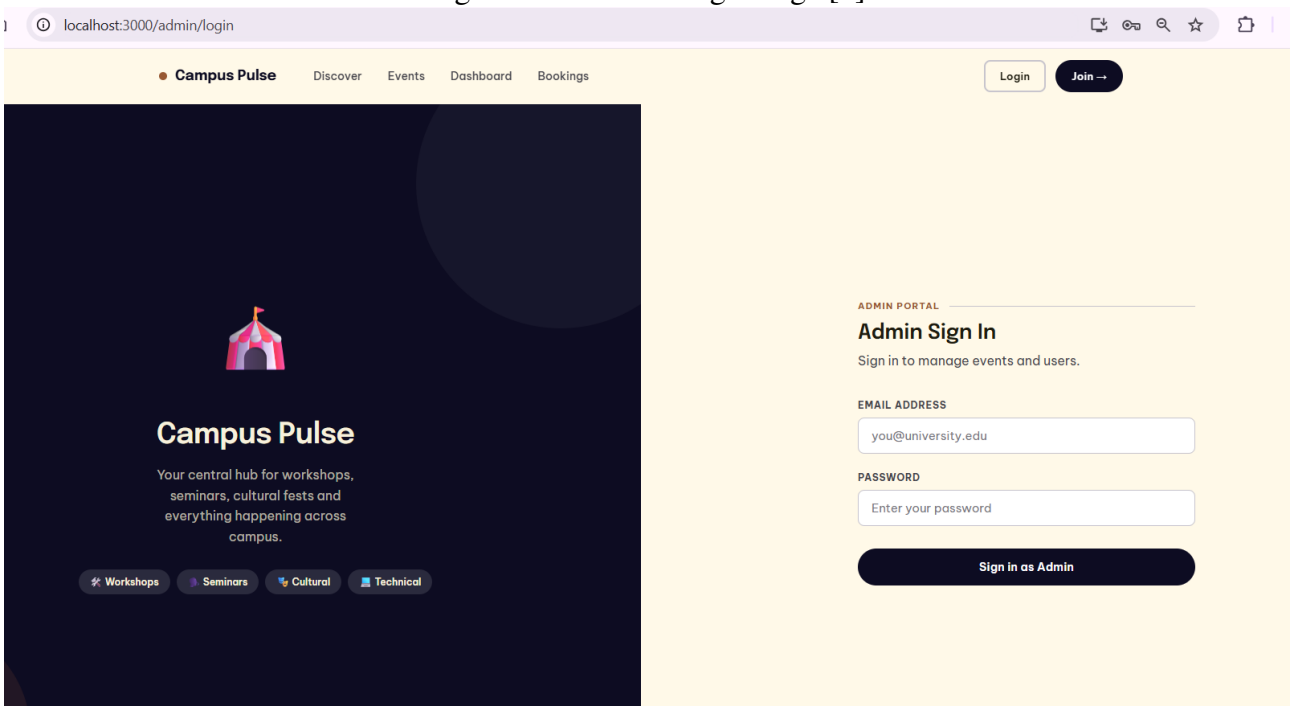

Figure 5.5: Secure login interface for administrators to access the event management dashboard.

Working

- Admin enters login credentials.
- The system verifies the information.
- On successful authentication, access to the dashboard is granted.

5.5.2 Event Creation Module

This module enables administrators to add new events to the system. Once created, these events become visible to users on the event listing page.

Figure 5.6: Event Creation by Admin [4]

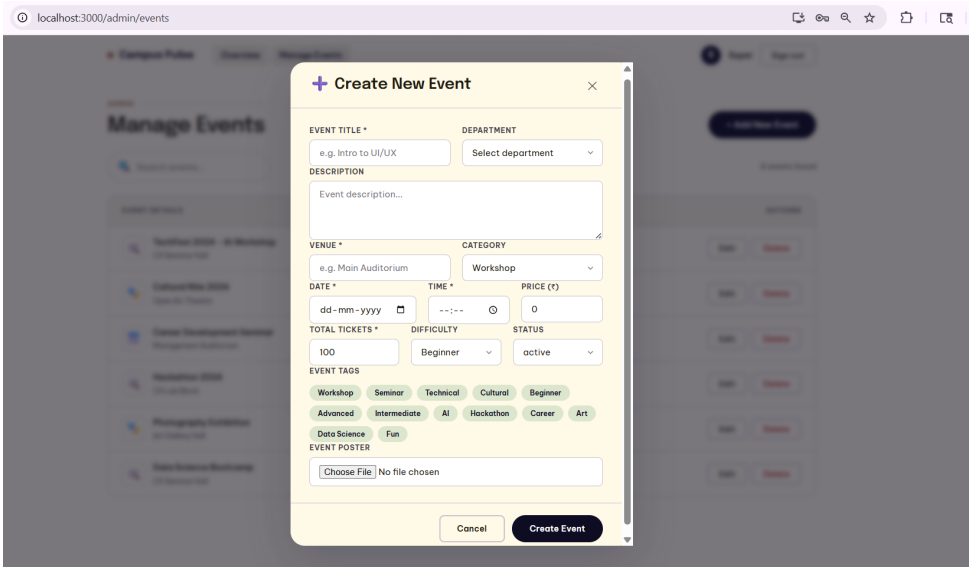

Figure 5.6: Admin interface for adding new events with details like name, date, venue, and capacity.

5.6 Testing Results

Table 5.1: Feature Testing Results [6]

Feature	**Status**	**Performance**
User Registration	Passed	Excellent
User Login	Passed	Excellent
Event Browsing	Passed	Excellent
Ticket Booking	Passed	Excellent
Admin Management	Passed	Very Good
Data Storage	Passed	Excellent
UI Responsiveness	Passed	Very Good

Table 5.1: This table summarizes the testing outcomes, indicating that all features passed with Excellent or Very Good performance.

5.7 Performance Metrics

Table 5.2: Application Performance [1]

Metric	**Value**
Page Load Time	$<$2 seconds
API Response Time	$<$500ms
Database Query Time	$<$100ms
Build Size (minified)	$\sim$250KB

Table 5.2: This table presents key performance metrics of the application, indicating fast response times, efficient database queries, and optimized build size.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The React JS-based web application for managing ticket bookings of internal department events has been successfully developed and implemented [1]. The system effectively fulfills the need for an automated and organized event management solution within academic environments.

The developed application offers:

1. An intuitive interface for browsing events and booking tickets
2. Secure user authentication and access control
3. Efficient data storage and management using MySQL [5]
4. Administrative functionalities for handling events and bookings
5. A flexible architecture that supports future scalability

Overall, the project showcases the effective use of modern full-stack technologies to streamline event management processes, minimize manual effort, and enhance user convenience.

6.2 Future Enhancements

The system can be further improved by incorporating the following features:

1. Integration of online payment systems
2. QR code-based ticket generation
3. Advanced analytics dashboard for event organizers
4. Development of a dedicated mobile application
5. Real-time alerts and notification system

Chapter 7

Addressing Complex Engineering Problem and SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP) [6]

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	React JS, hooks, validation, UI design knowledge [1].
WP2	Conflicting Requirements	Technical & non-technical constraints	WK4, WK5	Balances usability with booking constraints.
WP3	Depth of Analysis	Analytical & design thinking	WK3, WK4	Component design, state handling, updates.
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates.
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic.
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Students, faculty, organizers.
WP7	Interdependence	System-level integration	WK3, WK5	UI, state, validation integration.

Table 7.1: Maps the project against seven CEP attributes, justifying how each complex engineering criteria is addressed.

7.2 Mapping to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA) [1]

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS [5].
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation.
EA3	Innovation	Modern technologies	Hooks, conditional rendering.
EA4	Consequences	Impact on users	Simplifies booking.
EA5	Familiarity	Beyond standard	Dynamic real-time system.

Table 7.2: Maps the project against five CEA attributes, highlighting resource usage.

7.3 SDG Mapping

Table 7.3: SDG Mapping for the Project [12]

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Supports academic events.
SDG 9	Industry, Innovation and Infrastructure	Promotes React-based systems.
SDG 10	Reduced Inequalities	Accessible to all users.
SDG 11	Sustainable Cities and Communities	Encourages organized events.

Table 7.3: Aligns the project with four SDGs, showing contributions to education, innovation,and communities.

References

1. React Documentation (2024). *React Official Docs*. Available at: https://react.dev

2. Oracle Corporation (2024). *MySQL Reference Manual*. Available at: https://dev.mysql.com/doc/refman/en/

3. Node.js Foundation (2024). *Node.js Documentation*. Available at: https://nodejs.org/docs

4. Express.js Team (2024). *Express.js Documentation*. Available at: https://expressjs.com

5. W3Schools (2024). *MySQL Tutorial*. Available at: https://www.w3schools.com/mysql/

6. GeeksforGeeks (2024). *Full Stack Development*. Available at: https://www.geeksforgeeks.org

7. Mozilla Developer Network (2024). *Web APIs Documentation*. Available at: https://developer.mozilla.org

8. Cvent Inc. (2024). *Event Management Software*. Available at: https://www.cvent.com

9. Meetup Inc. (2024). *Meetup Platform*. Available at: https://www.meetup.com

10. CampusGroups (2024). *CampusGroups Platform*. Available at: https://www.campusgroups.com

11. Whova Inc. (2024). *Event Management Platform*. Available at: https://www.whova.com

12. United Nations (2024). *Sustainable Development Goals*. Available at: https://sdgs.un.org/goals